Week11 예습과제

6. 임베딩 | Word2Vec ~ 순환신경망

목차

- 6.1 Word2Vec | 분산 표현, CBOW, Skip-gram, 계층적 소프트맥스, 네거티브 샘플링
- 6.2 Skip-gram 모델 실습 | 임베딩, 데이터 전처리, 학습, 코사인 유사도
- 6.3 Gensim Word2Vec | 학습 매개변수와 유사 단어 추출
- 6.4 fastText | 하위 단어, 모델 실습, OOV 단어 처리
- 6.5 순환 신경망 | 은닉 상태, 입력·출력 구조, 양방향·다층 RNN
- 6.6 순환 신경망 클래스 | PyTorch RNN 구조와 텐서 차원
- 6.7 장단기 메모리(LSTM) | 게이트, 셀 상태, 은닉 상태
- 6.8 LSTM 모델 실습 | LSTM 출력 구조와 투사 계층
- 6.9 문장 분류 | SentenceClassifier, 데이터 전처리·학습·평가
- 6.10 사전 학습 임베딩 | Word2Vec 초기화 및 문장 분류 적용

6.1 Word2Vec

- **Word2Vec:** 주변 단어의 분포를 이용하여 단어의 의미적 관계를 벡터로 학습하는 단어 임베딩 방법
- **분포 가설(Distributional Hypothesis):** 비슷한 문맥에서 등장하는 단어들은 의미도 유사할 가능성이 높다는 가정
  - 단어가 문장 속에서 함께 등장하는 관계를 학습하여 단어마다 저차원 실수 벡터를 생성함
  - 각 단어를 서로 독립적인 원-핫 벡터로 표현하는 방식과 달리, 단어 사이 의미적 유사성을 수치적으로 반영 가능
- **분산 표현(Distributed Representation):** 의미 정보를 벡터의 여러 차원에 분산하여 나타내는 표현
  - 어휘 전체 개수만큼 차원이 커지는 원-핫 인코딩과 달리 비교적 작은 임베딩 차원 사용 가능
  - 단어 간 거리를 비교하거나 가까운 단어를 탐색하는 데 활용

6.1.1 단어 벡터화

표현 방식의 차이

- **희소 벡터:** 단어별 고유 위치를 사용하므로 어휘 크기만큼 차원이 증가함
- **밀집 벡터:** 의미 관계를 여러 실수 성분에 나누어 담아 벡터 간 비교 가능

구분	원-핫 표현	Word2Vec 밀집 표현
차원	어휘 사전 크기와 동일	설정한 임베딩 차원
값의 형태	한 성분만 1, 대부분 0	여러 성분이 학습된 실숫값
단어 관계	벡터 자체로 의미 유사도 표현 어려움	학습된 벡터의 방향·거리 비교 가능

- **희소 표현(Sparse Representation):** 대부분의 성분이 0 인 벡터. 원-핫 인코딩이 대표적임
- **밀집 표현(Dense Representation):** 대부분의 성분이 실숫값으로 구성되는 저차원 벡터. Word2Vec 에서 사용

표 6.3 단어의 희소 표현

소	0	1	0	0	0
잃고	1	0	0	0	0
외양간	0	0	0	1	0
고친다	0	0	0	0	1

표 6.4 단어의 밀집 표현

소	0.3914	−0.1749	…	0.5912	0.1321
잃고	−0.2893	0.3814	…	−0.1492	−0.2814
외양간	0.4812	0.1214	…	−0.2745	0.0132
고친다	−0.1314	−0.2809	…	0.2014	0.3016

- **임베딩 벡터:** 학습 과정에서 단어의 문맥을 반영하도록 매개변수 값이 갱신됨

– 단어가 5,000 개일 때 원-핫 벡터 차원은 5,000, 10 차원 밀집 벡터를 사용하면 단어당 10 개 실수로 표현 가능

– 벡터의 크기 자체보다 학습된 벡터 간 상대적 위치·유사성에 주목

6.1.2 CBOW (Continuous Bag of Words)

학습 데이터 구성

– 중심 단어를 지정한 뒤 윈도 범위에 속하는 주변 단어를 입력으로 추출함

– 주변 단어의 임베딩 정보를 결합하여 중심 단어의 확률을 높이는 방향으로 학습함

• **CBOW:** 주변 단어(Context Word)를 입력으로 사용하여 중심 단어(Center Word)를 예측하는 방식

– **입력:** 중심 단어 양옆에 있는 문맥 단어들 / 출력: 중심 단어가 될 확률

– **윈도(Window):** 중심 단어를 기준으로 문맥에 포함할 주변 단어 범위

– **슬라이딩 윈도(Sliding Window):** 중심 단어를 순차적으로 이동하면서 학습 쌍을 생성하는 과정

• CBOW 모델 구조

– 주변 단어들을 원-핫 벡터 등으로 입력 → 임베딩 계층(룩업 테이블)에서 벡터 획득

– 주변 단어의 임베딩 표현을 결합 → 투사 계층(Projection Layer) → 중심 단어 예측

– 학습 과정에서 중심 단어 예측에 유용하도록 임베딩 가중치가 조정됨

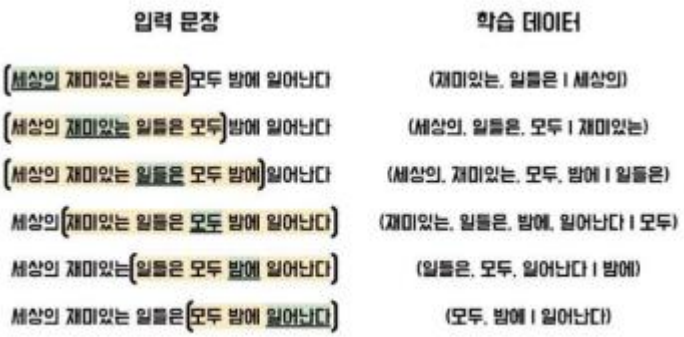


그림 6.4 CBoW의 학습 데이터 구성

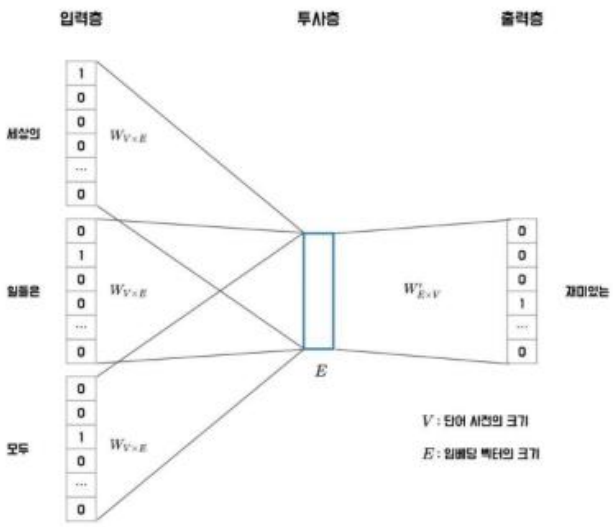


그림 6.5 CBoW 모델 구조

6.1.3 Skip-gram

학습 데이터 구성

– 중심 단어 하나를 입력으로 두고, 윈도 내부 각 주변 단어를 별도의 정답으로 취급함

– 중심 단어가 문맥 단어를 잘 예측하도록 임베딩 행렬과 출력 가중치를 갱신함

구분	CBOW	Skip-gram
입력	주변 단어들	중심 단어
정답	중심 단어	주변 단어 각각
훈련 쌍	문맥 묶음 → 중심 단어	중심 단어 → 문맥 단어

- **Skip-gram:** 하나의 중심 단어를 입력으로 사용하여 주변 단어들을 예측하는 방식
  – **입력:** 중심 단어 1 개 / 출력: 해당 단어의 문맥에 등장할 주변 단어들
  – CBOW 와 입력·출력 관계가 반대이며, 중심 단어마다 여러 학습 쌍이 만들어질 수 있음
- Skip-gram 모델 구조
  – 중심 단어 벡터를 임베딩 계층에 입력 → 은닉층 표현 생성 → 주변 단어의 분포 예측
  – 각 중심 단어에 대해 윈도 범위 내 주변 단어를 개별 예측 대상으로 사용
- **CBOW:** 주변 문맥 → 중심 단어 / Skip-gram: 중심 단어 → 주변 문맥

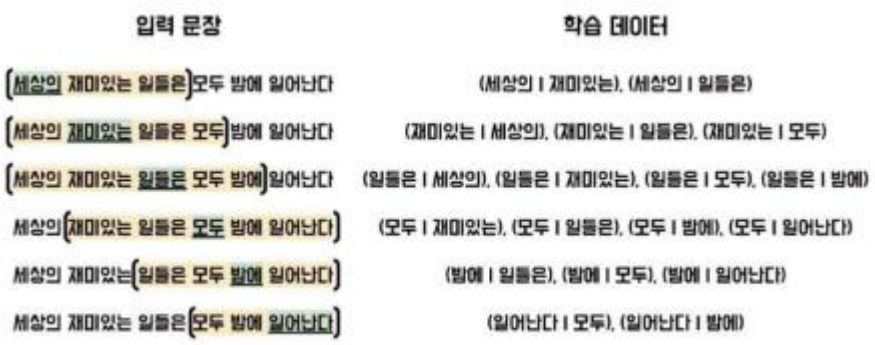


그림 6.6 Skip-gram의 학습 데이터 구성

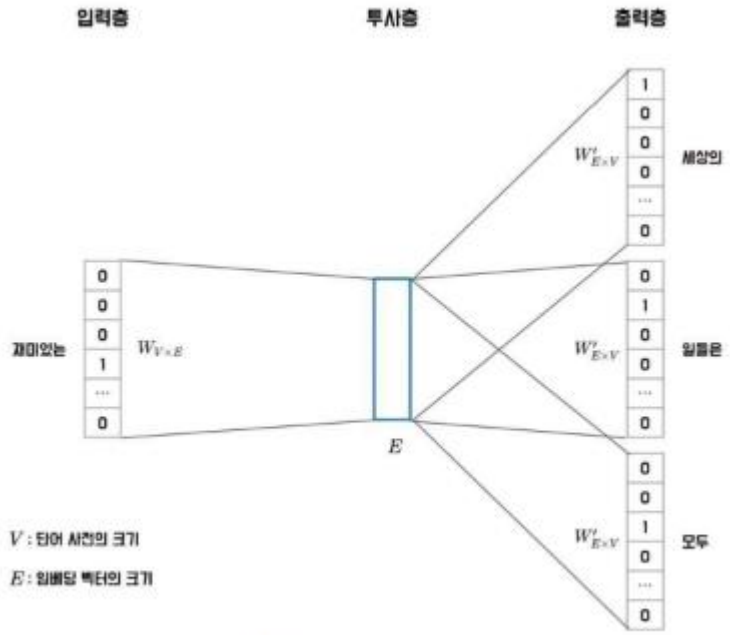


그림 6.7 Skip-gram 모델 구조

6.1.4 계층적 소프트맥스 (Hierarchical Softmax)

- 어휘의 각 단어를 이진 트리(Binary Tree)의 잎 노드(Leaf Node)에 대응시켜 출력 확률을 계산하는 방식
  - 전체 어휘에 대한 소프트맥스 확률을 한 번에 계산하지 않고, 루트에서 단어의 잎 노드까지 도달하는 경로의 확률을 계산함
  - 각 내부 노드에서 왼쪽·오른쪽 가지를 선택하는 확률을 이용해 목표 단어의 확률 결정

– 일반 소프트맥스 계산량 O(V)와 비교하여 트리 경로를 활용할 경우 O(log V) 수준으로 감소 가능

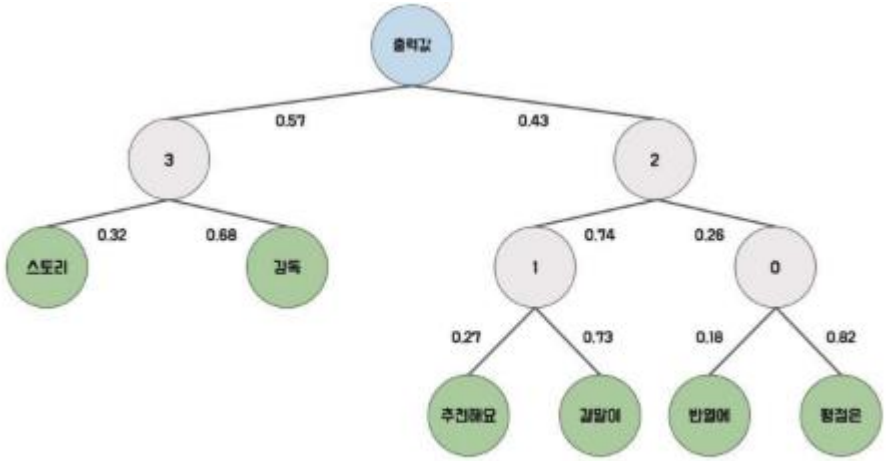


그림 6.8 계층적 소프트맥스 구조

6.1.5 네거티브 샘플링 (Negative Sampling)

양성·음성 예시의 차이

- **양성 예시:** 실제 윈도 내에 등장하여 함께 학습할 중심 단어–주변 단어 쌍
- **음성 예시:** 단어 빈도 분포를 기준으로 추출한, 실제 문맥과 무관한 비교 단어
- 양성 쌍의 점수는 높이고 음성 쌍의 점수는 낮추는 방향으로 매개변수 최적화

• 실제 문맥에 함께 등장한 단어를 양성(Positive) 예시로, 등장하지 않은 단어를 음성(Negative) 예시로 구성해 학습

- 전체 어휘에 대한 출력 확률을 계산하는 대신 일부 양성·음성 예시만 선별하여 손실 계산
- 무작위 추출된 단어가 많더라도 실제 문맥 관계가 없는 단어의 점수는 낮아지도록 학습

• **네거티브 샘플링 확률:** 단어의 등장 빈도를 기반으로 음성 단어를 추출

- 자주 등장하는 단어의 출현 확률을 조정해 음성 샘플 선택 확률 계산
- 학습 단어 쌍마다 소수의 음성 예시만 사용하므로 출력 계층의 계산량을 절감함

수식 6.9 네거티브 샘플링의 추출 확률

$$P(w_i) = \frac{f(w_i)^{0.75}}{\sum_{j=0}^{V} f(w_j)^{0.75}}$$

입력 데이터	출력 데이터
재미있는	세상의
재미있는	일들은
재미있는	모두

입력 데이터	출력 데이터
재미있는, 세상의	1
재미있는, 일들은	1
재미있는, 모두	1
재미있는, 걸어가다	0
재미있는, 주십시오	0
재미있는, 미리	0
재미있는, 안녕하세요	0

그림 6.9 네거티브 샘플링 모델의 훈련 데이터

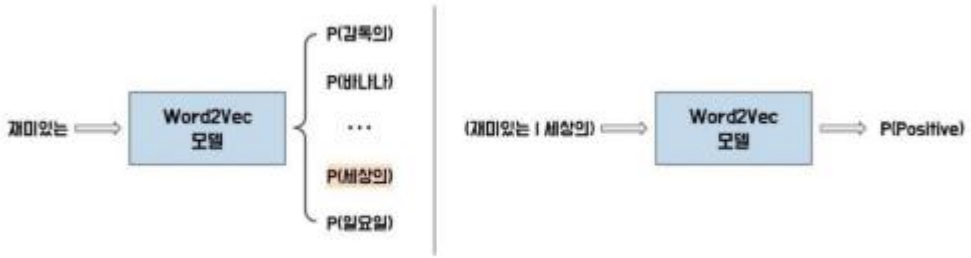


그림 6.10 소프트맥스 Word2Vec과 네거티브 샘플링 Word2Vec

6.2 모델 실습: Skip-gram

6.2.1 임베딩 클래스 (p.286)

- **torch.nn.Embedding:** 정수 단어 ID 를 해당 단어의 임베딩 벡터로 변환하는 PyTorch 계층
  - **num_embeddings:** 임베딩할 전체 단어의 수(어휘 사전 크기)
  - **embedding_dim:** 각 단어의 임베딩 벡터 차원
  - **padding_idx:** 패딩 토큰의 인덱스 지정; 해당 토큰의 임베딩 처리에 사용
  - **max_norm:** 임베딩 벡터 노름 상한 지정 / norm_type: 노름 계산 방식 지정
- **실습 설정:** 전체 단어 수 5,000 개, 임베딩 차원 128 을 사용하는 Skip-gram 모델 구성

임베딩 클래스

```
embedding = torch.nn.Embedding(
    num_embeddings,
    embedding_dim,
    padding_idx=None,
    max_norm=None,
    norm_type=2.0
)
```

6.2.2 VanillaSkipgram 클래스 (예제 6.3 | p.287)

- nn.Module 을 상속받아 Skip-gram 신경망 구현
  - **self.embedding = nn.Embedding(...):** 중심 단어의 정수 ID 를 128 차원 벡터로 변환
  - **self.linear = nn.Linear(...):** 임베딩 표현을 전체 어휘 크기의 출력 점수로 변환
  - **forward(input_ids):** 입력 단어 ID → 임베딩 → 선형 계층 → 로짓(logits) 반환

6.2.3 데이터 전처리 (예제 6.4~6.8 | p.288~290)

전처리 순서

  - 영화 리뷰 로딩 → Okt 형태소 토큰화 → 빈도 기반 어휘 사전 → 윈도별 (중심, 주변) 단어 쌍 → 정수 인덱스 변환

- **Korpora.load("nsmc"):** 네이버 영화 리뷰 말뭉치(NSMC)를 불러옴
  - **pandas.DataFrame(corpus.test):** 테스트 데이터셋을 표 형태로 구성
  - **Okt().morphs(review):** 영화 리뷰 문장을 형태소 단위로 토큰화
- **build_vocab():** 출현 빈도를 기반으로 어휘 사전 구성
  - **Counter():** 토큰별 출현 횟수 계산 / most_common(): 빈도가 높은 토큰 순서대로 추출
  - **n_vocab=5000:** 빈도 기준 상위 단어 수 제한 / special_tokens: 특수 토큰을 사전에 추가
  - **token_to_id:** 단어 → 정수 ID / id_to_token: 정수 ID → 단어 매핑
- **get_word_pairs(tokens, window_size):** 중심 단어와 주변 단어의 학습 쌍 생성
  - 각 문장에서 단어 위치를 순회하고, 중심 단어 인덱스 전후 window_size 범위의 문맥 단어 추출
  - 중심 단어와 문맥 단어를 (center_word, context_word) 튜플로 저장
- **get_index_pairs():** 단어 문자열로 된 쌍을 사전의 정수 인덱스 쌍으로 변환
  - 사전에 없는 단어는 특수 토큰 <unk>로 대응

6.2.4 텐서·데이터로더 및 모델 설정 (예제 6.8~6.9 | p.291)

- **TensorDataset:** 중심 단어 인덱스와 주변 단어 인덱스를 묶어 학습 데이터셋 생성
- **DataLoader:** 학습 데이터를 미니 배치로 나누고 순서를 섞어 모델에 제공
  - batch_size=32 / shuffle=True 설정으로 배치 학습 수행
- **device:** CUDA 사용 가능 시 GPU, 아니면 CPU 사용
- VanillaSkipgram(vocab_size, embedding_dim=128): 모델 생성 후 device 로 이동
- **CrossEntropyLoss():** 전체 어휘에 대한 예측 로짓과 실제 주변 단어 인덱스로 손실 계산
- **SGD(learning rate=0.1):** 오차 역전파 결과를 반영해 모델 매개변수 갱신

6.2.5 Skip-gram 모델 학습 (예제 6.10 | p.292)

학습 단계

- ① input_ids 에서 중심 단어의 임베딩을 조회함
- ② linear 계층으로 어휘 사전 크기만큼 로짓을 산출함
- ③ target_ids 에 해당하는 주변 단어와 로짓 사이의 교차 엔트로피 손실을 계산함
- ④ zero_grad → backward → step 순으로 가중치를 갱신함
- ⑤ 10 개 에포크의 평균 손실을 출력하여 감소 추이를 확인함

- **학습 단계:** 배치 로드 → 중심 단어 임베딩 → 주변 단어 점수 계산 → 손실 → 역전파 → 가중치 갱신
  - **zero_grad():** 기존 기울기 초기화 / backward(): 기울기 계산 / step(): 가중치 업데이트
  - **epoch:** 전체 학습 데이터를 한 번 모두 사용한 주기.
  - 배치별 손실을 누적한 뒤 데이터로더 길이로 나누어 에포크 평균 손실 확인

6.2.6 임베딩 결과 및 코사인 유사도 (예제 6.11~6.12 | p.293~294)

- **word2vec.embedding.weight:** 학습 완료된 단어 임베딩 가중치 행렬
  - **detach().cpu().numpy():** 기울기 추적을 중단하고 NumPy 배열로 변환
  - **token_to_embedding:** 단어 문자열을 임베딩 벡터로 매핑해 조회
- **코사인 유사도(Cosine Similarity):** 벡터 사이 각도의 코사인을 사용해 의미적 유사성 비교
  - cosine_similarity(a,b) = (a·b) / (||a|| × ||b||)
  - 값이 1 에 가까울수록 벡터 방향이 유사함을 의미. -1 에 가까우면 반대 방향, 0 부근은 직교에 해당
- **top_n_index():** 기준 단어와 코사인 유사도가 높은 단어들의 인덱스 검색
  - 유사도 행렬을 계산하고 argsort()로 상위 단어 순위를 구하는 방식

예제 6.12 단어 임베딩 유사도 계산

```
import numpy as np
from numpy.linalg import norm

def cosine_similarity(a, b):
    cosine = np.dot(b, a) / (norm(b, axis=1) * norm(a))
    return cosine

def top_n_index(cosine_matrix, n):
    closest_indexes = cosine_matrix.argsort()[::-1]
    top_n = closest_indexes[1 : n + 1]
    return top_n


cosine_matrix = cosine_similarity(token_embedding, embedding_matrix)
top_n = top_n_index(cosine_matrix, n=5)

print(f"{token}와 가장 유사한 5 개 단어")
for index in top_n:
    print(f"{id_to_token[index]} - 유사도 : {cosine_matrix[index]:.4f}")
```

출력 결과

```
연기와 가장 유사한 5 개 단어
연기력 - 유사도 : 0.3976
배우 - 유사도 : 0.3167
시나리오 - 유사도 : 0.3130
악마 - 유사도 : 0.2977
까지도 - 유사도 : 0.2892
```

6.3 모델 실습: Gensim Word2Vec

6.3.1 Gensim 라이브러리 (p.295)

- **Gensim:** Word2Vec 등 단어 임베딩 모델의 학습과 활용을 지원하는 Python 라이브러리
  - 교재 실습에서는 Word2Vec 클래스를 사용하여 자체 신경망을 직접 구현하지 않고 모델 생성·학습 가능
  - 단어 간 유사성, 단어 벡터 조회 및 모델 저장·불러오기 기능 제공

6.3.2 Word2Vec 주요 인자 (p.296~297)

학습 방식 관련 인자

- sg=0 은 CBOW, sg=1 은 Skip-gram 사용
- hs=1 은 계층적 소프트맥스 사용; negative 는 음성 샘플 수를 지정

어휘·학습량 관련 인자

- min_count 로 저빈도 단어 제외, max_final_vocab 으로 최대 어휘 수 제한
- window 로 문맥 범위, vector_size 로 임베딩 차원, epochs 로 반복 횟수 설정
- **sentences:** 학습 문장 모음. 각 문장을 토큰 리스트 형태로 입력
- **corpus_file:** 학습에 사용할 말뭉치 파일 경로
- **vector_size:** 단어 임베딩 차원.
- **alpha:** 학습률 / min_alpha: 학습률 최소값
- **window:** 중심 단어 주변에서 고려할 문맥 윈도 크기
- **min_count:** 학습에 포함할 단어의 최소 출현 빈도
- **workers:** 병렬 학습에 사용할 작업자 수
- **sg:** 0 은 CBOW, 1 은 Skip-gram 선택
- **hs:** 계층적 소프트맥스 적용 여부
- **negative:** 네거티브 샘플링에 사용할 음성 예시 수
- **ns_exponent:** 음성 단어 추출 분포의 빈도 지수
- **epochs:** 전체 학습 데이터 반복 횟수
- **batch_words:** 작업 단위의 단어 수 / max_final_vocab: 최종 어휘 사전 크기 상한

6.3.3 Word2Vec 학습 및 조회 (예제 6.13~6.14 | p.297~298)

- Word2Vec(sentences=tokens, vector_size=128, window=5, min_count=1, epochs=3, ...): 토큰화된 데이터를 이용해 단어 벡터 학습
  - **save(), load():** 모델 파일 저장 및 재사용
- **word2vec.wv[word]:** 해당 단어의 학습된 벡터 조회
- **word2vec.wv.most_similar(word, topn=5):** 특정 단어와 가장 유사한 단어와 점수 반환
  - 학습 데이터 규모, 매개변수, 초기화에 따라 이웃 단어와 유사도 결과가 달라질 수 있음

6.4 fastText

6.4.1 fastText 의 특징 (p.299~300)

하위 단어 생성 과정

- ① 단어 양끝에 <, > 경계 기호를 부착함
- ② 정한 길이의 문자 N-gram 으로 단어 내부를 나눔
- ③ 하위 단어 임베딩을 결합하여 전체 단어 표현에 활용함

Word2Vec 과의 차이

  – 단어 단위 학습뿐 아니라 내부 철자 정보를 반영함 → 학습 시 보지 못한 단어도 일부 하위 문자열을 통해 표현 가능

- **fastText:** 단어 자체를 하나의 단위로만 취급하지 않고 하위 단어(Subword) 정보를 활용해 임베딩을 학습하는 모델
  – Word2Vec 의 단어 단위 학습 방식에서 확장하여 단어 내부의 부분 문자열 정보 반영
- **하위 단어:** 단어를 문자 N-gram 등 더 작은 단위의 조합으로 분해한 표현
  – 단어 앞뒤에 경계 표시(<, >)를 부착하여 전체 단어의 시작과 끝 구별
  – 설정된 N-gram 길이에 따라 부분 문자열을 추출하고, 해당 부분들의 벡터를 이용하여 단어 벡터 구성
- **장점:** 학습 말뭉치에 등장하지 않은 미등록 단어(OOV)도 부분 문자열 정보로 벡터 추정 가능
  – Word2Vec 에서는 사전에 없는 단어의 임베딩을 직접 조회하기 어렵지만 fastText 에서는 하위 단어를 조합할 수 있음

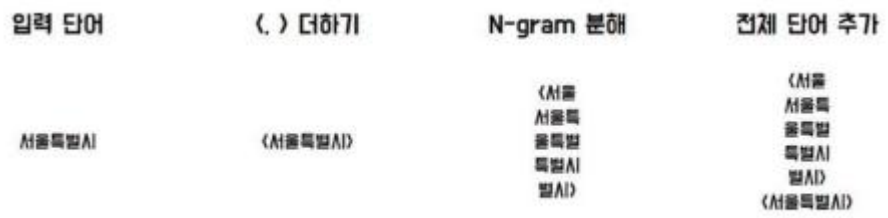


그림 6.11 fastText의 하위 단어 집합

6.4.2 fastText 클래스 주요 인자 (p.301~302)

- **Gensim 의 FastText:** Word2Vec 과 유사한 인자로 구성되며 하위 단어 관련 매개변수 추가
  – **vector_size:** 임베딩 차원 / window: 문맥 윈도 / sg: Skip-gram 사용 여부
  – **min_n, max_n:** 단어에서 생성할 문자 N-gram 의 최소·최대 길이
  – **bucket:** 부분 문자열 특징을 저장하는 해시 버킷 크기
  – **min_count:** 최소 출현 빈도 / epochs: 학습 반복 수

6.4.3 KorNLI 를 활용한 학습 (예제 6.15~6.16 | p.302~303)

- **Korpora.load("kornli"):** KorNLI 말뭉치 로드
  – 전제(premise)와 가설(hypothesis) 문장 쌍을 학습 말뭉치로 활용
  – **get_all_texts(), get_all_pairs():** 말뭉치의 문장 데이터 획득
- **sentence.split():** 문장을 공백 기준으로 분리해 학습용 토큰 리스트 구성
- FastText(sentences=tokens, vector_size=128, ...): 모델 학습
  – 문자 N-gram 크기 설정 및 문맥 윈도를 포함하여 fastText 벡터 학습

  - **save(), load():** 모델 저장·복원.

6.4.4 OOV 단어 조회 (예제 6.17 | p.304)

- **oov_token:** 학습 어휘 사전에 등록되지 않은 단어
  - **oov_token in fasttext.wv.key_to_index:** 단어가 학습 어휘에 존재하는지 확인
- **fasttext.wv[oov_token]:** 하위 단어 벡터를 조합하여 해당 문자열의 벡터 생성 가능
- **fasttext.wv.most_similar(oov_token, topn=5):** 미등록 단어와 가까운 단어 탐색
  - 오탈자·변형된 단어처럼 일부 문자열을 공유하는 표현에서 하위 단어 방식의 장점 확인

6.5 순환 신경망 (Recurrent Neural Network, RNN)

6.5.1 순차 데이터와 순환 구조 (p.304~306)

순환 계산의 흐름

  - ① 현재 입력 x_t 와 직전 은닉 상태 h_(t−1)를 함께 사용함
  - ② 공유 가중치로 현재 은닉 상태 h_t 를 갱신함
  - ③ 새로운 은닉 상태를 다음 시점으로 전달하면서 시퀀스를 순차 처리함

- **순환 신경망:** 순서와 시간에 따라 연결된 데이터(Sequence Data)를 처리하기 위해 이전 시점의 은닉 상태를 다음 시점에 전달하는 신경망
  - 단어·문장·음성·시계열 데이터처럼 입력 순서가 의미에 영향을 미치는 데이터에 활용
- **Time Step:** 순차 데이터의 각 시점 / Hidden State: 이전 시점까지 입력된 정보를 요약한 은닉 상태
  - 현재 입력 x_t 와 이전 은닉 상태 h_(t1)을 결합하여 새로운 은닉 상태 h_t 계산
- **순환 계산:** h_t = σ_t(W_h h_(t1) + W_x x_t + b_h)
  - **σ_t:** 은닉 상태에 적용되는 활성화 함수 / W_h: 이전 은닉 상태의 가중치
  - **W_x:** 현재 입력의 가중치 / b_h: 편향
- **출력 계산:** o_t = σ_o(h_t)를 거쳐 시점별 출력을 생성하는 방식 제시
  - **교재 수식 6.11~6.12 (p.306~307):** 은닉 상태 업데이트와 출력 계산

$$y_t = \sigma_y(h_t)$$

$$y_t = \sigma_y(W_y h_t + b_y)$$

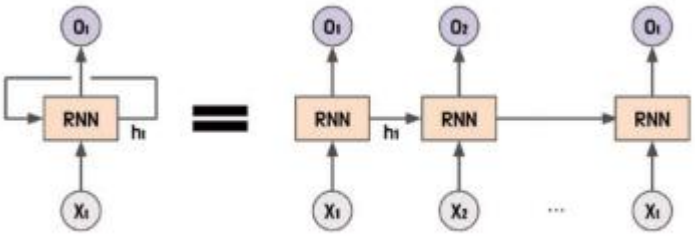


그림 6.12 순환 신경망의 셀

6.5.2 입력·출력 형태에 따른 구조 (p.307~309)

형태	입력	출력	적용 예시
일대다	단일 입력	여러 시점	이미지 캡셔닝
다대일	입력 시퀀스	단일 결과	영화 리뷰 감성 분류
다대다	입력 시퀀스	출력 시퀀스	Seq2Seq 문장 변환

- **일대다(One-to-Many):** 단일 입력을 받아 여러 시점에 출력 생성. 예: 이미지 설명 생성(Image Captioning)
- **다대일(Many-to-One):** 입력 시퀀스를 처리한 뒤 하나의 출력 생성. 예: 감성 분석(Sentiment Analysis)
- **다대다(Many-to-Many):** 입력 시퀀스에 대해 여러 출력 생성. 예: Seq2Seq, 인코더-디코더 기반 문장 변환
  - 과제 목적에 따라 전체 시점의 출력 또는 마지막 은닉 상태를 선택하여 사용함

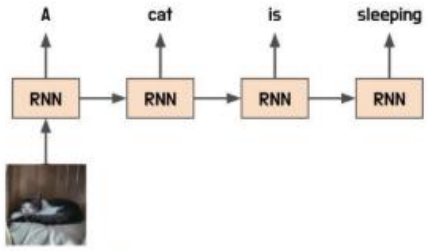


그림 6.13 순환 신경망의 일대다 구조

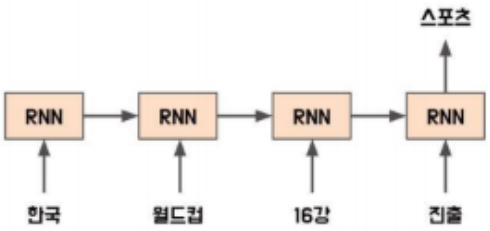


그림 6.14 순환 신경망의 다대일 구조

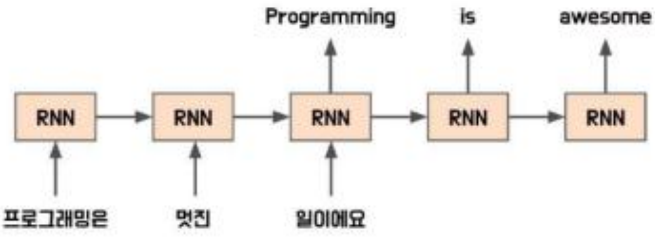


그림 6.15 순환 신경망의 다대다 구조

6.5.3 양방향 순환 신경망 (Bidirectional RNN)

- 입력 시퀀스를 정방향과 역방향으로 각각 처리하여 두 방향의 문맥을 반영하는 순환 구조
  - **정방향 은닉 상태:** 처음 → 마지막 시점 / 역방향 은닉 상태: 마지막 → 처음 시점

– 같은 시점에서 양쪽 방향의 정보를 합치거나 연결하여 출력 표현 생성

- 문장 내 특정 단어를 해석할 때 앞쪽 문맥뿐 아니라 뒤쪽 문맥도 활용 가능

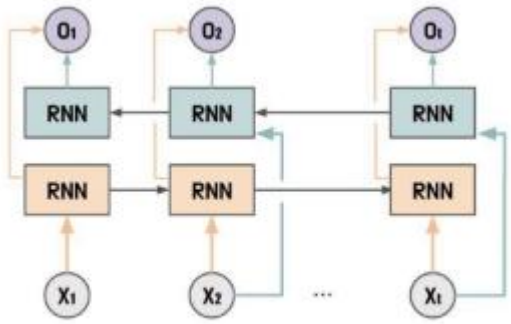


그림 6.16 양방향 순환 신경망 구조

6.5.4 다층 순환 신경망 (Stacked RNN)

- 여러 개의 순환 계층을 수직으로 연결하여 이전 계층의 은닉 상태를 다음 계층 입력으로 사용하는 구조
  – 각 계층은 시간 방향으로 순환 계산 수행 / 상위 계층으로 갈수록 더 복잡한 순차 특징 학습 가능
  – 계층 수가 증가하면 표현력 향상 가능하지만 계산량·학습 난도도 증가할 수 있음

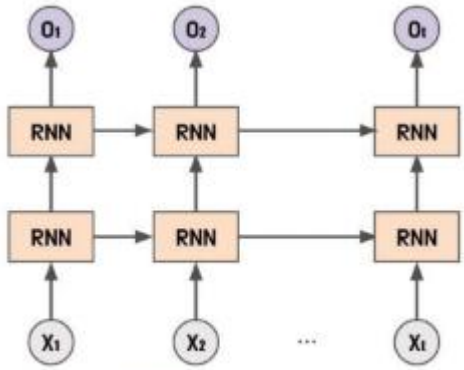


그림 6.17 다중 순환 신경망 구조

6.6 순환 신경망 클래스 (p.311~313)

6.6.1 torch.nn.RNN 주요 인자

– **input_size:** 각 시점에서 입력되는 특징 벡터 차원

– **hidden_size:** RNN 의 은닉 상태 벡터 크기

– **num_layers:** 층을 쌓는 순환 계층 개수

– **nonlinearity:** 은닉 상태 활성화 함수.

  - **bias:** 편향 사용 여부
  - **batch_first:** 입력 텐서의 배치 차원을 첫 번째에 둘지 결정
  - **dropout:** 다층 모델의 계층 사이에 적용할 드롭아웃 비율
  - **bidirectional:** 양방향 RNN 적용 여부

6.6.2 양방향 RNN 실습 (예제 6.18 | p.312~313)

텐서 차원 해석

  - batch_first=True 이므로 입력 순서는 [배치, 시점, 특징]임
  - 양방향에서 출력 특징 차원은 hidden_size×2 이며, 마지막 은닉 상태의 첫 차원은 계층 수×2 임

- **입력 설정:** input_size=128, output_size=256, num_layers=3, bidirectional=True
  - **입력 텐서:** (batch_size, sequence_len, input_size) 형태를 사용
  - **outputs:** 각 시점에서 계산된 마지막 RNN 계층의 은닉 상태 출력 시퀀스
  - **hidden:** 각 계층·방향의 마지막 은닉 상태
- **교재 예시:** batch_size=4, sequence_len=6 → outputs 크기 (4, 6, 512)
  - 양방향이므로 hidden_size=256 × 방향 수 2 = 512
- **hidden 크기:** (num_layers × 방향 수, batch_size, hidden_size) = (6, 4, 256)
  - 순환 신경망의 결과 크기는 층 수·방향 수·배치 크기 설정에 따라 달라짐

6.7 장단기 메모리 (Long Short-Term Memory, LSTM)

6.7.1 LSTM 의 필요성 (p.313~314)

기본 RNN 과 LSTM

  - **기본 RNN:** 과거 정보를 은닉 상태에 반복 반영하므로 긴 시퀀스에서 오래전 정보가 약화될 수 있음
  - **LSTM:** 셀 상태를 별도로 유지하고 세 게이트로 정보의 보존·갱신·출력을 제어함

구성	역할	값의 주요 범위
망각 게이트 f_t	과거 셀 상태의 보존 정도	0~1
입력 게이트 i_t	새 후보 정보의 반영 비율	0~1
후보 상태 g_t	새로 기록할 정보	–1~1
출력 게이트 o_t	현재 은닉 상태의 출력 비율	0~1

- 기본 RNN 은 시점이 길어질수록 앞선 정보의 영향이 줄어들어 장기 의존성(Long-Term Dependency)을 학습하기 어려울 수 있음
  - **장기 의존성:** 오래전 입력된 정보가 나중의 출력에 영향을 주는 관계

– 장기 시퀀스 학습에서 기울기 소실 또는 기울기 폭주가 나타날 수 있음

- **LSTM:** 장기 정보를 유지하는 셀 상태(Cell State)와 필요한 정보의 흐름을 제어하는 게이트를 도입한 순환 구조
  – **셀 상태 C_t:** 시점 간에 장기 정보를 전달하는 경로 / 은닉 상태 h_t: 현재 시점의 출력 표현
  – 망각 게이트, 입력 게이트, 출력 게이트로 정보 삭제·추가·출력을 제어

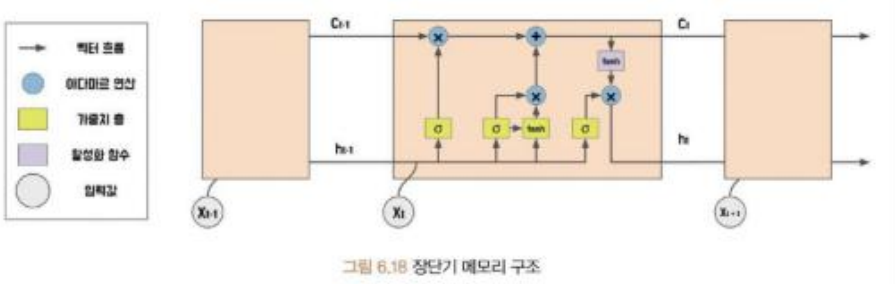


그림 6.18 장단기 메모리 구조

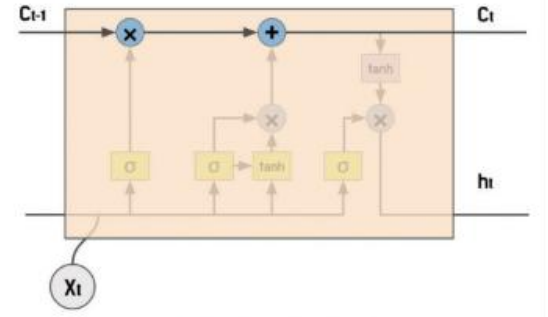


그림 6.19 장단기 메모리 셀 상태

6.7.2 망각 게이트 (Forget Gate)

- **망각 게이트:** 이전 시점의 셀 상태 중 현재 시점까지 유지할 정보의 비율 결정
- f_t = σ(W_f x_t + W_hf h_(t1) + b_f)
  – 시그모이드 함수의 출력 범위 0~1. 0 에 가까운 값은 해당 정보 삭제, 1 에 가까운 값은 유지
  – 계산된 f_t 를 이전 셀 상태 C_(t1)에 요소별 곱하여 필요한 정보만 보존

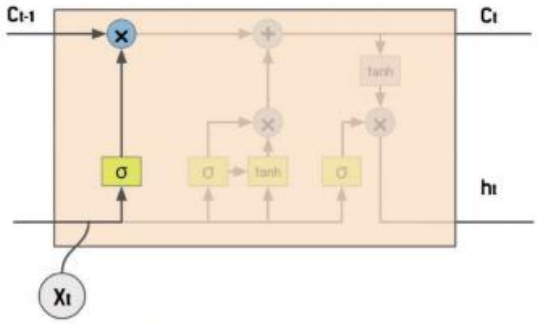


그림 6.20 장단기 메모리의 망각 게이트

6.7.3 입력 게이트 (Input Gate) 및 후보 셀 상태

- **입력 게이트:** 현재 입력에서 새롭게 셀 상태에 기록할 정보의 양 조절
- **g_t = tanh(W_g x_t + W_hg h_(t1) + b_g):** 새로 기록할 후보 정보 생성
- **i_t = σ(W_i x_t + W_hi h_(t1) + b_i):** 후보 정보를 반영할 비율 계산
  - tanh 의 출력 범위 -1~1 / 시그모이드의 출력 범위 0~1
- **셀 상태 업데이트:** C_t = f_t ⊙ C_(t1) + i_t ⊙ g_t
  - ⊙**(Hadamard Product):** 벡터 또는 텐서의 같은 위치 원소끼리 곱하는 연산
  - 이전 셀 상태에서 유지할 정보와 현재 시점의 새 정보가 결합됨

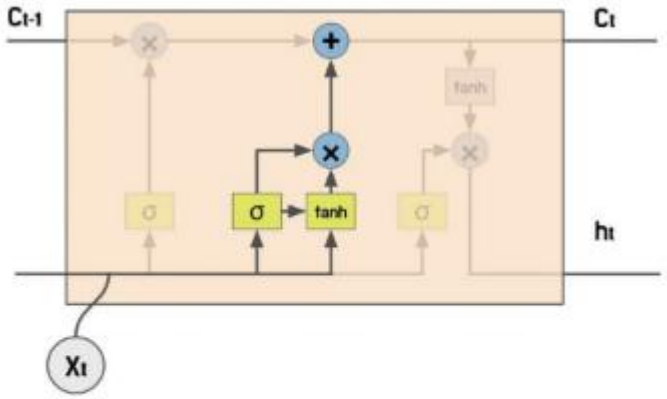


그림 6.21 장단기 메모리의 기억 게이트

수식 6.14 기억 게이트

$$g_t = tanh(W_x^{(g)} x_t + W_x^{(g)} h_{t-1} + b^{(g)})$$
$$i_t = sigmoid(W_x^{(i)} x_t + W_x^{(i)} h_{t-1} + b^{(i)})$$

수식 6.15 메모리 셀

$$c_t = f_t \odot c_{t-1} + g_t \odot i_t$$

6.7.4 출력 게이트 (Output Gate)

- **출력 게이트:** 갱신된 셀 상태 중 외부로 내보낼 은닉 상태의 정보 선택
- o_t = σ(W_o x_t + W_ho h_(t1) + b_o)
- h_t = o_t ⊙ tanh(C_t)
  - 출력 게이트 비율과 현재 셀 상태의 tanh 변환 결과를 요소별 곱하여 은닉 상태 생성
  - 셀 상태는 장기 정보 보존, 은닉 상태는 현재 시점의 출력과 다음 시점 계산에 활용

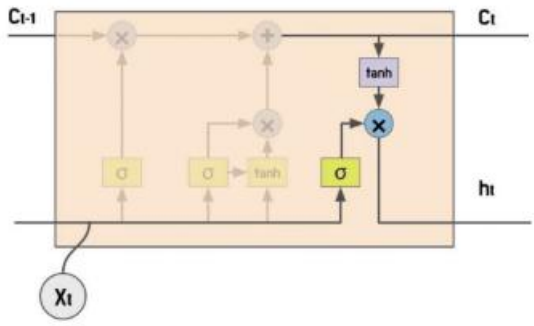


그림 6.22 장단기 메모리의 출력 게이트

수식 6.16 출력 게이트

$$o_t = \sigma(W_x^{(o)} x_t + W_h^{(o)} h_{t-1} + b^{(o)})$$

수식 6.17 은닉 상태 갱신

$$h_t = o_t \odot tanh(c_t)$$

6.8 장단기 메모리 클래스 (p.320~322)

6.8.1 torch.nn.LSTM 주요 인자

- **input_size:** 각 시점 입력 벡터의 차원
- **hidden_size:** 은닉 상태 및 셀 상태의 크기
- **num_layers:** LSTM 계층 개수
- **bias:** 편향 포함 여부
- **batch_first:** batch, sequence, feature 순서 사용 여부
- **dropout:** 계층 사이의 드롭아웃 비율
- **bidirectional:** 양방향 LSTM 사용 여부
- **proj_size:** LSTM 출력에 적용할 투사 계층 차원. 0 이면 투사하지 않음

6.8.2 양방향 LSTM 실습 (예제 6.19 | p.321~322)

출력과 상태의 크기 구분

- **outputs:** 각 시점에 해당하는 마지막 층의 은닉 출력 전체
- **h_n:** 각 계층·방향의 마지막 은닉 상태, 투사 크기가 지정되면 proj_size 기준
- **c_n:** 각 계층·방향의 마지막 셀 상태, hidden_size 기준

- **입력 텐서:** (batch_size, sequence_len, input_size) = (4, 6, 128)
  - **h_0:** 초기 은닉 상태 / c_0: 초기 셀 상태
  - 투사 차원 proj_size 가 0 보다 크면 은닉 출력 크기는 hidden_size 대신 proj_size 를 따름
- **outputs:** (4, 6, 128) — 양방향 × 투사 크기 64

- **h_n:** (6, 4, 64) — 3 계층 × 2 방향, 배치 4, 투사 크기 64
- **c_n:** (6, 4, 256) — 셀 상태는 hidden_size 256 차원 유지
  - **출력 결과가 h_n 과 c_n 에서 다른 이유:** 은닉 상태에는 투사 계층이 적용되지만 셀 상태의 내부 크기는 유지되기 때문

6.9 모델 실습: 문장 분류

6.9.1 실습 모델 구조 (p.323~325)

- **문장 분류(Sentence Classification):** 문장 전체를 입력받아 긍정·부정 등의 범주를 예측하는 작업
- **실습 구성:** 단어 임베딩 → RNN/LSTM → 마지막 시점 출력 → Dropout → Linear 분류기
  - 입력 문장의 각 토큰 ID 를 임베딩 벡터로 바꾼 뒤 순차 모델에서 문맥 정보 학습
  - 마지막 시점의 은닉 상태를 문장 전체의 대표 특징으로 활용
  - **분류기의 출력 클래스 수:** 2 개 (긍정·부정)

6.9.2 SentenceClassifier (예제 6.20 | p.324~325)

모델 계산 흐름

  - 토큰 ID → nn.Embedding → RNN/LSTM → 마지막 시점 표현 → Dropout → Linear → 로짓
  - 이진 분류 시 마지막 출력의 차원은 1 개이며, sigmoid 와 BCEWithLogitsLoss 를 활용하여 학습 가능

- **SentenceClassifier(nn.Module):** 임베딩 계층과 RNN 또는 LSTM 을 조합하여 만든 문장 분류 모델
  - **model_type:** RNN 과 LSTM 중 사용할 순차 모델 유형 결정
  - **nn.Embedding:** 어휘 ID 를 밀집 벡터로 변환
  - **nn.RNN / nn.LSTM:** 임베딩 시퀀스에서 순서에 따른 문맥 특징 학습
  - bidirectional=True 시 양방향 특징 사용, 분류기 입력 차원이 두 배가 됨
  - **nn.Dropout:** 학습 시 일부 특징을 무작위로 제외하여 과적합을 완화
  - **nn.Linear:** 마지막 은닉 표현을 하나의 로짓으로 변환하여 긍정·부정 이진 분류 수행

6.9.3 영화 리뷰 데이터 전처리 (예제 6.21~6.23 | p.325~328)

패딩과 미등록 단어 처리

  - **<pad>:** 문장 길이를 통일하기 위해 추가하며, 실제 단어 정보가 아닌 자리 채움에 사용함
  - **<unk>:** 어휘 사전에 등록되지 않은 단어를 하나의 특별한 ID 로 변환함
  - 길이 32 를 넘는 문장은 잘라 내고, 부족한 문장은 패딩을 붙여 배치 처리 가능하게 함

- **Korpora.load("nsmc"):** 네이버 영화 리뷰 데이터셋 사용
  - train / test 를 pandas DataFrame 으로 변환하여 text 와 label 확인

- Okt 를 이용하여 영화 리뷰 문장을 형태소 단위로 분할
- build_vocab(corpus, n_vocab=5000, special_tokens=["<pad>", "<unk>"]): 어휘 사전 생성
  - **<pad>:** 입력 시퀀스의 길이를 맞추는 패딩 토큰 / <unk>: 사전에 없는 단어 대체
  - **token_to_id, id_to_token:** 단어 ID 변환을 위한 매핑 딕셔너리
- **pad_sequences():** 문장의 토큰 수가 달라도 일정한 길이의 정수 시퀀스로 변환
  - **max_length=32:** 최대 시퀀스 길이. 길면 자르고 짧으면 패딩 추가
  - 정수 ID 배열을 만들고 학습·평가용 입력 시퀀스로 저장

6.9.4 학습용 텐서·데이터로더 구성 (예제 6.24 | p.329)

- **torch.tensor():** 정수 ID 시퀀스와 문장 레이블을 텐서 형태로 변환
  - 입력은 단어 인덱스이므로 정수형 텐서 사용 / 레이블은 이진 분류용 실수형 변환
- **TensorDataset:** 리뷰 시퀀스와 해당 레이블을 한 데이터 단위로 묶음
- **DataLoader:** 학습용과 테스트용 데이터를 각각 배치 단위로 제공

6.9.5 모델·손실 함수·최적화기 (예제 6.25 | p.329~330)

- **n_vocab=len(token_to_id):** 실제 학습 데이터에서 구성된 어휘 크기 사용
- hidden_dim=64, embedding_dim=128, n_layers=2: 모델 주요 하이퍼파라미터
- **device:** GPU 를 사용할 수 있으면 CUDA, 아니면 CPU 선택
- **criterion=nn.BCEWithLogitsLoss():** 이진 분류 로짓에 사용되는 손실 함수
  - 시그모이드 계산과 이진 교차 엔트로피 계산을 결합한 손실 함수
- **optimizer=optim.RMSprop(...):** 모델 파라미터 업데이트를 위한 최적화 알고리즘

6.9.6 학습 및 평가 함수 (예제 6.26 | p.330~332)

학습·평가의 차이

  - **학습:** model.train() 적용 후 역전파와 optimizer.step()으로 가중치를 갱신함
  - **평가:** model.eval(), torch.no_grad()를 사용해 가중치 갱신 없이 예측 결과를 측정함
  - **정확도 계산:** 로짓에 sigmoid 를 적용한 확률을 0.5 기준으로 이진 예측한 뒤 실제 라벨과 비교함
- **train():** 학습 모드 전환 후 미니 배치마다 손실·역전파·가중치 갱신 수행
  - **model.train():** 학습 모드 활성화 / optimizer.zero_grad(): 기울기 초기화
  - **logits:** 문장에 대한 분류 점수 / criterion(logits, labels): 학습 손실
  - **loss.backward(), optimizer.step():** 기울기 역전파 및 매개변수 업데이트
- **test():** 평가 모드에서 손실과 정확도 계산

  - **model.eval():** 평가 모드 / torch.no_grad(): 기울기 계산 없이 추론
  - **sigmoid(logits) > 0.5:** 긍정 클래스 예측 여부를 이진값으로 변환
  - 예측 레이블과 실제 레이블을 비교해 정확도(Accuracy) 계산
- **교재 실습:** epochs=5, interval=500 설정 후 매 에포크 학습·평가 수행
  - 평가 출력은 손실 및 정확도 추이를 확인하는 용도로 활용

6.10 사전 학습 임베딩 활용

6.10.1 학습된 임베딩 조회 (예제 6.27 | p.333)

- 학습된 SentenceClassifier 의 임베딩 가중치 추출
  - classifier.embedding.weight.detach().cpu().numpy(): 텐서 형태의 임베딩 행렬 조회
  - 단어와 대응하는 벡터를 딕셔너리로 만들어 특정 단어의 학습 결과 확인
- 모델의 목적과 학습 데이터 규모에 따라 학습된 벡터의 성질이 달라짐

6.10.2 외부 Word2Vec 임베딩 초기화 (예제 6.28 | p.333)

- 앞서 학습한 Gensim Word2Vec 모델을 불러와 문장 분류 모델의 임베딩 초기값으로 활용
  - **Word2Vec.load(...):** 저장된 임베딩 모델 불러오기
  - **np.zeros((n_vocab, embedding_dim)):** 어휘 사전 크기와 임베딩 차원에 맞는 초기 임베딩 행렬 구성
  - token_to_id 에 포함된 각 단어를 검사하고 Word2Vec 학습 어휘에 존재할 경우 해당 벡터를 복사
- **nn.Embedding.from_pretrained():** 기존 임베딩 가중치를 PyTorch 임베딩 계층으로 초기화
  - 학습되지 않은 단어는 0 벡터 또는 기존 초기화 방식 적용 가능

6.10.3 SentenceClassifier 클래스 변경 (예제 6.29 | p.334)

사전 학습 가중치 사용 조건

  - pretrained_embedding 이 제공되면 from_pretrained 로 가중치를 초기화함
  - 전달되지 않으면 기존의 nn.Embedding 을 새로 생성함
  - 동일한 분류 구조를 유지한 상태에서 임베딩 초기화 방식의 차이를 비교함
- pretrained_embedding 매개변수를 추가하여 사전 학습 임베딩을 선택적으로 전달받도록 클래스 수정
  - **매개변수가 None 이 아닌 경우:** from_pretrained()로 사전 학습 임베딩 초기화
  - **매개변수가 None 인 경우:** 일반 nn.Embedding 으로 랜덤 초기화
- 그 외 RNN/LSTM 및 분류기 구조는 동일하게 유지

6.10.4 사전 학습 임베딩 적용 및 비교 (예제 6.30 | p.334~335)

- 사전 학습 벡터를 이용하여 SentenceClassifier 를 생성하고 동일한 학습·평가 함수를 적용
  - 선행 학습에서 얻은 단어 의미 정보를 분류 작업에 활용하여 시작점 개선 가능
- **실습 의미:** 별도 임베딩 모델에서 학습된 단어 표현을 문장 분류 모델의 초기 표현으로 전이해 활용
  - 단, 데이터 일치 여부와 사전 학습 품질에 따라 성능 향상 정도는 달라질 수 있음